COMP 511

Computer Programming Foundations II

Array and LinkedList

Dr. Judy Luo

Arrays

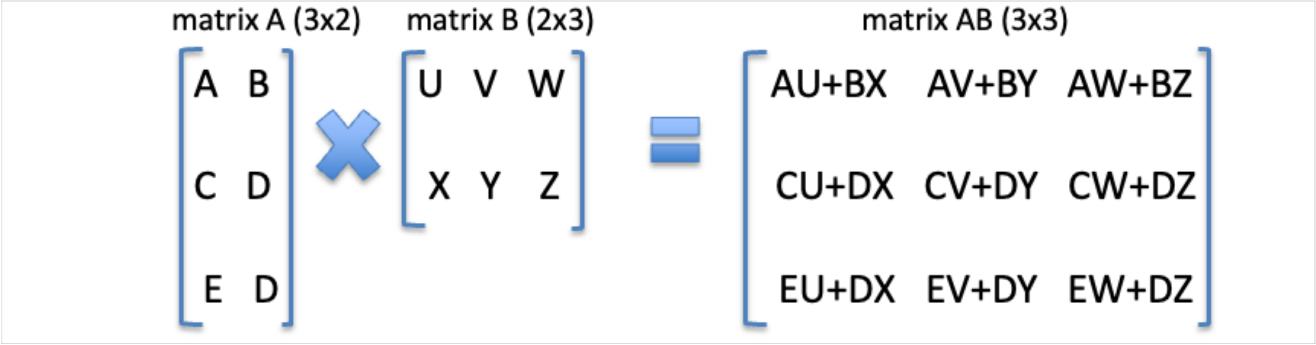
- An *array* is the basic mechanism for storing a collection of identically typed entities.
- Array element accessed by index. The number of elements is the length of the array.
- Declaration of array =>
    - By size: `int [ ] array2 = new int [ 5 ];`
    - By content: `int [ ] array3 = { 3, 4, 10, 6 };`

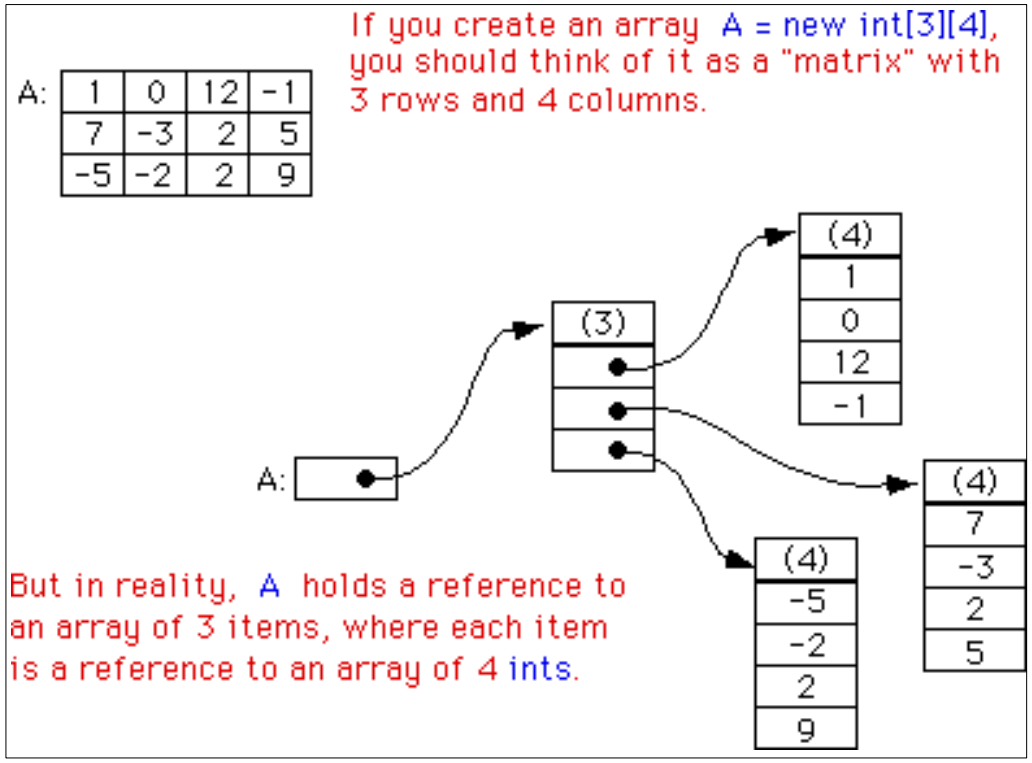
Array Limitations

- Fixed size
- Adding or removing from middle is hard

Multidimensional Arrays

- A *multidimensional array* is an array that is accessed by more than one index
- Example: int [ ][ ] x = new int[2][3];
- A common example of this is a matrix





Multidimensional Arrays Example

```java
public class twoDimArray {
    public static void main(String args[]){
        //creating two matrices
        int a[][]={{3,2,1},
                {6,5,4},
                {9,8,7}};
        int b[][]={{10,11,12},
                {20,21,22},
                {30,31,32}};

        //creating another matrix to store the multiplication of two matrices
        int c[][]=new int[3][3];  //3 rows and 3 columns

        //multiplying and printing multiplication of 2 matrices
        for(int i=0;i<3;i++){
            for(int j=0;j<3;j++){
                c[i][j]=0;
                for(int k=0;k<3;k++)
                {
                    c[i][j]+=a[i][k]*b[k][j];
                }
                System.out.print(c[i][j]+" ");  //printing matrix element
            }
            System.out.println();//new line
        }
    }
}
```

Output

```
100 106 112
280 295 310
460 484 508
```

ArrayLists

- The ArrayList is used for expanding arrays.
- ArrayList<String> array = new ArrayList<String>( );

	Array	ArrayList
Resizable	No	Yes
Iterating values	for, for each	Iterator , for each
Performance	Fast	Slow in comparison
Multidimensional	Yes	No
Add Elements	Assignment operator	Add method
Length	length variable	size method

```java
import java.util.ArrayList;
import java.util.Iterator;
public class ArrayList519 {
    public static void main(String[] args) {

        // ArrayList Example
        ArrayList<String> arrlistobj = new ArrayList<String>();
        arrlistobj.add("INFS 519");
        arrlistobj.add("Program Design and Data Structures\n\n");
        Iterator it = arrlistobj.iterator();
        System.out.print("ArrayList object output :\n");
        while(it.hasNext())
            System.out.print(it.next() + " ");


        // Array Example
        String[] arrayobj = new String[2];
        arrayobj[0]= "INFS 519";
        arrayobj[1]= "Program Design and Data Structures ";
        System.out.print("Array object output :\n");
        for(int i=0; i < arrayobj.length ;i++)
            System.out.print(arrayobj[i] + " ");
    }
}
```

Java Example for Array and ArrayList

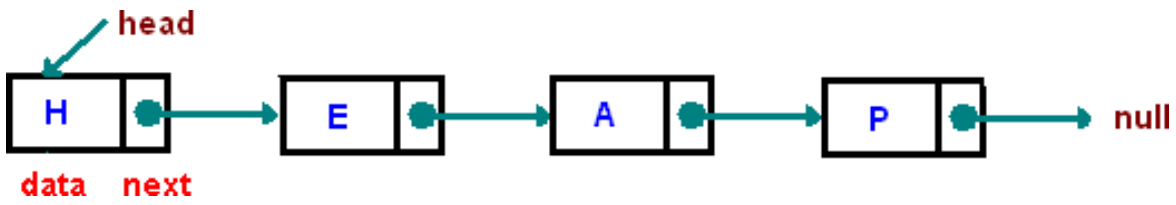
Output

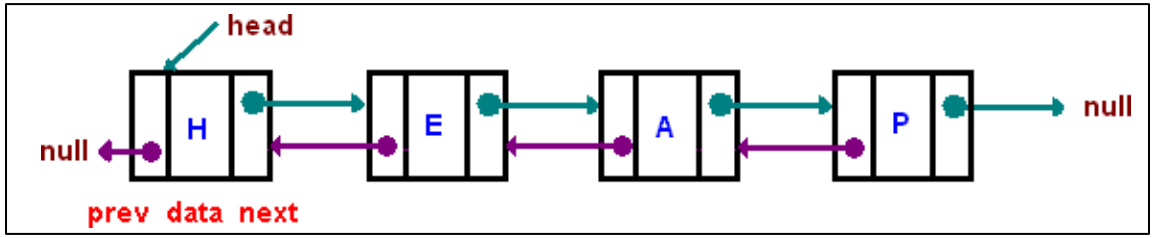
```
ArrayList object output :
INFS 519 Program Design and Data Structures

 Array object output :
INFS 519 Program Design and Data Structures
```

LinkedList

- A data structure that is used to avoid large amounts of data movement.
- store each object in a *node* that contains the object and a reference to the next node in the list





LinkedList VS ArrayList ?

LinkedList Example

```java
import java.util.*;
public class LinkedList519 {
    public static void main (String[] args){
        /* Linked List Declaration */
        LinkedList<String> linkedlist = new LinkedList<String>();

        /*add(String Element) adds elements to the linked list*/
        linkedlist.add("Item1");
        linkedlist.add("Item5");
        linkedlist.add("Item3");
        linkedlist.add("Item6");
        linkedlist.add("Item2");

        /*Display Linked List Content*/
        System.out.println("Linked List Content: " +linkedlist);

        /*Add First and Last Element*/
        linkedlist.addFirst("First Item");
        linkedlist.addLast("Last Item");
        System.out.println("LinkedList after addition: " +linkedlist);

        /* get and set Values */
        System.out.println("First element: " +linkedlist.get(0));
        linkedlist.set(0, "Changed first item");

        System.out.println("First element after update by set method:" + linkedlist.get(0));
        /*Remove first and last element*/
        linkedlist.removeFirst();
        linkedlist.removeLast();
        System.out.println("After deletion of first and last element: " +linkedlist);

        /* Add to a Position and remove from a position*/
        linkedlist.add(0, "Newly added item");
        linkedlist.remove(2);
        System.out.println("Final Content: " +linkedlist);
    }
}
```

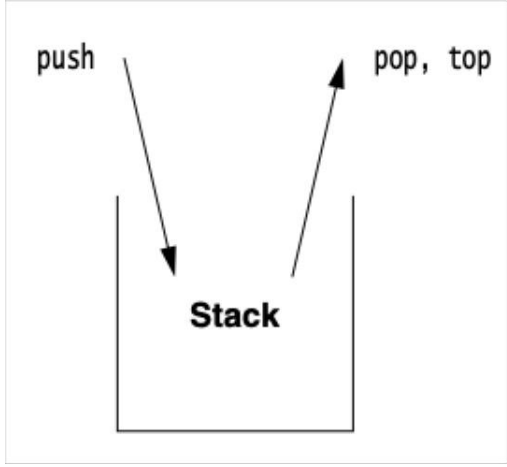
Output

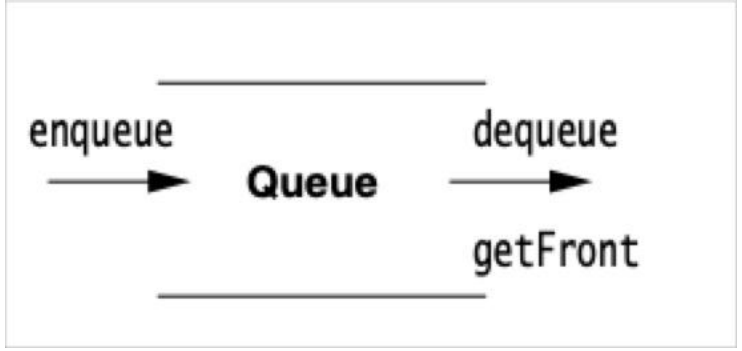
```
Linked List Content: [Item1, Item5, Item3, Item6, Item2]
LinkedList after addition: [First Item, Item1, Item5, Item3, Item6, Item2, Last Item]
First element: First Item
First element after update by set method:Changed first item
After deletion of first and last element: [Item1, Item5, Item3, Item6, Item2]
Final Content: [Newly added item, Item1, Item3, Item6, Item2]
```

Stacks and Queues

Stack is data structure in which access is restricted to the most recently inserted item

Last In First Out (LIFO)



Queue is data structure in which access is restricted to the least recently inserted item

First In First Out (FIFO)